

HealthOS Initial PostgreSQL Architecture — Corrected Baseline

1. Architecture Style

HealthOS will begin as a modular monolith using one PostgreSQL database and a small number of domain-oriented schemas:

```
healthos_db
├── identity
├── organization
├── platform
├── care
├── financial
└── governance
```

Create later, only when the product requires them:

```
operations
analytics
ai
```

Possible future schema extraction:

```
platform.commercial_*    → commercial.*
platform.workflow_*      → workflow.*
governance.compliance_*  → compliance.*
operations.notification_* → notification.*
operations.integration_*  → integration.*
```

The early schema grouping reduces operational complexity for a 2–3 person team while preserving logical domain boundaries through table names and application modules.

2. Identifier and Common-Column Standard

2.1 Major Entities

Use a dual-identifier model for externally addressable entities:

id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
uuid UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE

Use:

id

for internal foreign keys and joins.

Use:

uuid

for:

- APIs
- frontend URLs
- webhooks
- external integrations
- cross-database references

Examples of major entities:

organization

facility

user_account

person

patient

practitioner

staff_member

appointment

encounter

prescription

bill
payment
feature
workflow_definition

2.2 Internal Child Entities

Internal child tables may use only:

id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

Examples:

prescription_item
bill_item
payment_allocation
feature_assignment
workflow_assignment
staff_assignment

Add a UUID later only when a child record must be independently exposed outside the backend.

2.3 Foreign Keys

Use **BIGINT** foreign keys:

patient_id BIGINT NOT NULL
REFERENCES identity.patient(id)

Do not use UUID foreign keys for normal internal joins.

2.4 Shared Columns

Mutable tables should normally include:

created_at TIMESTAMPTZ NOT NULL DEFAULT now()
updated_at TIMESTAMPTZ

Tenant-owned tables must include:

organization_id BIGINT NOT NULL

Facility-specific tables should include:

facility_id BIGINT

Use **TIMESTAMPZ** and store timestamps in UTC.

Use **status** initially instead of implementing soft deletion on every table.

Sequence gaps are normal.

Incremental IDs must not be used as:

- invoice numbers
- receipt numbers
- medical record numbers
- token numbers
- legal business sequence numbers
- other business identifiers

3. Logto Responsibility Boundary

Logto owns:

authentication
passwords
MFA
sessions
OIDC/OAuth
JWT issuance
Logto organizations
generic roles
generic permissions
role assignments

HealthOS owns:

internal user mapping

tenant and facility context
patient identity
practitioner qualification
staff employment
staff facility assignment
market and package configuration
feature entitlement
workflow configuration
clinical records
billing
consent
clinical and operational audit

HealthOS must not store:

password
password_hash
MFA secret
refresh token
session
generic role
generic permission
generic user-role assignment

4. **identity** Schema

Purpose:

- map Logto users into HealthOS
- store tenant-scoped human profiles
- store patient profiles
- store practitioner qualifications

identity

├── user_account
├── person
├── patient
└── practitioner

4.1 identity.user_account

Represents a Logto-authenticated account inside HealthOS.

This is a global authentication mapping and is not owned by one hospital.

id	
uuid	
external_subject	Logto JWT "sub"
email	nullable
display_name	nullable
status	
last_login_at	nullable
created_at	
updated_at	

Constraints:

UNIQUE(external_subject)

Do not use email as the primary identity key.

The Logto **sub** claim is the authoritative external identity.

A Logto user can work in multiple organizations.

Therefore, **user_account** does not contain:

organization_id
person_id
facility_id
role
permission

The relationship to an organization-specific staff or patient profile is stored in:

organization.staff_member
identity.patient

4.2 identity.person

Represents a human profile inside one HealthOS organization.

```
id
uuid
organization_id
first_name
last_name
date_of_birth      nullable
gender             nullable
phone              nullable
email              nullable
status
created_at
updated_at
```

A tenant-scoped person model avoids automatically sharing personal data between unrelated hospitals.

The same real-world human may have separate person records in separate organizations.

This is intentional because each organization owns its own operational copy of the person's data.

4.3 identity.patient

Represents the patient role of a person.

```
id
uuid
organization_id
person_id
user_account_id    nullable
primary_facility_id nullable
medical_record_number
status
created_at
updated_at
```

Constraints:

UNIQUE(organization_id, medical_record_number)

UNIQUE(organization_id, person_id)

`user_account_id` is optional because many patients will not use the patient application.

The same Logto account may be linked to patient records in more than one organization.

Example:

Logto user



HealthOS user_account



Patient record at Hospital A

Patient record at Clinic B

4.4 `identity.practitioner`

Represents a clinically qualified healthcare professional.

id

uuid

organization_id

person_id

registration_number

registration_authority nullable

specialization nullable

status

created_at

updated_at

Constraints:

UNIQUE(organization_id, registration_number)

UNIQUE(organization_id, person_id)

This table answers:

Is this person clinically qualified?
What is the registration number?
What is the specialization?

It does not answer where the person currently works.

Employment and facility assignment belong to the **organization** schema.

A receptionist or front-desk operator does not need a practitioner record.

5. **organization** Schema

Purpose:

- tenant management
- hospital and clinic hierarchy
- staff employment
- facility and department assignment

organization

- organization
- facility
- department
- staff_member
- staff_assignment

5.1 **organization.organization**

Represents the HealthOS tenant.

id
uuid
logto_organization_id
market_id
name
organization_type
status
created_at
updated_at

Examples of **organization_type**:

clinic
hospital
diagnostic_center
pharmacy

Constraints:

UNIQUE(logto_organization_id)

The Logto organization controls:

IAM membership
generic RBAC
organization context in tokens

The HealthOS organization controls:

healthcare business data
facilities
staff employment
market assignment
subscription
features
workflows
clinical data
billing

5.2 **organization.facility**

Represents a physical or virtual operating location.

id
uuid
organization_id
code
name

timezone
status
created_at
updated_at

Constraint:

UNIQUE(organization_id, code)

Examples:

Main Clinic
Andheri Branch
Pune Hospital
Online Consultation Facility

5.3 organization.department

Represents a department inside a facility.

id
uuid
organization_id
facility_id
code
name
status
created_at
updated_at

Constraint:

UNIQUE(facility_id, code)

Examples:

Reception
Cardiology
General Medicine

Billing
Pharmacy
Laboratory

This table may remain unused for small clinics.

5.4 organization.staff_member

Represents a person's employment or operational relationship with an organization.

id
uuid
organization_id
person_id
user_account_id nullable
employee_code nullable
designation
employment_type nullable
status
joined_at nullable
left_at nullable
created_at
updated_at

Constraints:

UNIQUE(organization_id, person_id)
UNIQUE(organization_id, employee_code)
WHERE employee_code IS NOT NULL

Examples of staff members:

front-desk operator
receptionist
hospital administrator
billing executive
accountant
lab technician
pharmacist
nurse

doctor
data-entry operator

designation is operational or HR information.

It must not be treated as the authorization role.

Generic roles and permissions remain in Logto.

Front-Desk Operator

A front-desk operator normally has:

identity.person
identity.user_account
organization.staff_member
organization.staff_assignment

A front-desk operator normally does not have:

identity.practitioner

Doctor

A doctor may have:

identity.person
identity.user_account
identity.practitioner
organization.staff_member
organization.staff_assignment

Visiting Practitioner Without Login

A visiting practitioner may have:

identity.person
identity.practitioner
organization.staff_member

and may not yet have:

identity.user_account

5.5 organization.staff_assignment

Defines where a staff member is allowed to operate.

```
id
staff_member_id
facility_id
department_id      nullable
is_primary
status
valid_from         nullable
valid_until        nullable
created_at
updated_at
```

Recommended constraint:

UNIQUE(staff_member_id, facility_id, department_id)

This table supports:

- one receptionist assigned to one clinic
- one doctor assigned to multiple facilities
- one accountant serving several branches
- temporary department assignments
- one administrator managing multiple facilities

Logto answers:

Does this user have patient:create permission?

HealthOS answers:

- Is this staff member active in this organization?
- Is this staff member assigned to this facility?
- Is this assignment currently valid?

Both checks are required.

6. platform Schema

Purpose:

- country and market configuration
- market-specific packages
- subscriptions
- feature entitlement
- configurable workflows

platform

```
|— country
|— market
|— feature
|— feature_assignment
|— commercial_package
|— commercial_subscription
|— workflow_definition
|— workflow_assignment
```

6.1 platform.country

id
uuid
iso_code
name
currency_code
default_timezone
status

Constraint:

UNIQUE(iso_code)

Examples:

IN
AE
US
GB

Country stores broad regional information.

Market stores the actual HealthOS product configuration.

6.2 platform.market

Represents a deployable HealthOS product configuration.

id
uuid
country_id
code
name
config JSONB
status
created_at
updated_at

Constraint:

UNIQUE(code)

Examples:

IN_CLINIC
IN_HOSPITAL
IN_DIAGNOSTIC
IN_ABDM_HOSPITAL
UAE_CLINIC
UAE_HOSPITAL

Example configuration:

{

```
"locale": "en-IN",  
"date_format": "DD/MM/YYYY",  
"allow_walk_in": true  
}
```

A market is more specific than a country.

One country can contain several HealthOS product configurations.

Example:

```
India  
├── IN_CLINIC  
├── IN_HOSPITAL  
├── IN_DIAGNOSTIC  
└── IN_ABDM_HOSPITAL
```

6.3 platform.commercial_package

Represents a market-specific subscription package.

```
id  
uuid  
market_id  
code  
name  
price  
currency_code  
billing_interval  
limits JSONB  
status  
created_at  
updated_at
```

Constraint:

UNIQUE(market_id, code)

Example limits:

```
{
  "max_users": 10,
  "max_facilities": 1,
  "ai_requests_per_month": 0
}
```

Examples:

```
IN_STARTER
IN_GROWTH
IN_ENTERPRISE
IN_ENTERPRISE_AI
```

Packages should normally be market-specific.

Avoid creating a universal global package filled with country-specific conditions.

6.4 platform.commercial_subscription

Assigns a package to an organization.

```
id
uuid
organization_id
package_id
status
starts_at
ends_at          nullable
created_at
updated_at
```

An organization should normally have only one active subscription.

Enforce this later using a partial unique index for active subscription statuses.

Example conceptual constraint:

```
CREATE UNIQUE INDEX uq_active_subscription_per_org
ON platform.commercial_subscription (organization_id)
WHERE status IN ('active', 'trial');
```

6.5 platform.feature

Master feature registry.

id
uuid
code
name
default_enabled
status
created_at
updated_at

Constraint:

UNIQUE(code)

Examples:

patient_registration
appointment_booking
queue_management
ai_soap
ai_transcription
abha_linking
abdm_record_sharing
fhir_export
telemedicine
inventory
insurance_verification

Feature codes should remain stable because they may be used by:

- backend authorization
- frontend rendering
- workflow definitions
- subscriptions
- usage metering
- integrations

6.6 platform.feature_assignment

Enables, disables, or configures a feature at one scope.

id

feature_id

market_id nullable

package_id nullable

organization_id nullable

facility_id nullable

enabled

config JSONB

effective_from nullable

effective_until nullable

created_at

updated_at

Exactly one scope must be populated:

```
CHECK (  
    num_nonnulls(  
        market_id,  
        package_id,  
        organization_id,  
        facility_id  
    ) = 1  
)
```

Resolution order:

Feature default

↓

Market

↓

Package

↓

Organization

↓

Facility

The most specific active assignment wins.

Example:

India clinic market	→ AI SOAP permitted
Starter package	→ AI SOAP disabled
Enterprise package	→ AI SOAP enabled
Hospital A	→ AI SOAP enabled
Hospital A Facility 2	→ AI SOAP disabled

Final resolution:

Hospital A Main Facility	→ enabled
Hospital A Facility 2	→ disabled
Starter Clinic B	→ disabled
Enterprise Hospital C	→ enabled

The backend must enforce feature availability.

Hiding a frontend component is not sufficient.

6.7 platform.workflow_definition

Stores a versioned workflow definition.

id
uuid
code
name
version_number
definition JSONB
status
created_at
updated_at

Constraint:

UNIQUE(code, version_number)

Example:

```
{
  "steps": [
    {
      "key": "patient_search",
      "component": "patient.search",
      "required": true
    },
    {
      "key": "patient_registration",
      "component": "patient.registration",
      "required": false
    },
    {
      "key": "appointment_booking",
      "component": "appointment.booking",
      "required": true
    }
  ],
  "transitions": [
    {
      "from": "patient_search",
      "to": "patient_registration",
      "condition": "patient_not_found"
    }
  ]
}
```

The frontend owns the actual React components.

The database controls:

- which steps appear
- step order
- step configuration
- transition rules

The frontend may maintain a component registry:

patient.search
patient.registration
appointment.booking
clinical.vitals
clinical.soap
billing.checkout

The database should not contain executable frontend code.

It should contain stable component keys and configuration.

6.8 platform.workflow_assignment

Assigns a workflow version to one scope.

id
workflow_definition_id

market_id	nullable
package_id	nullable
organization_id	nullable
facility_id	nullable

is_default
config_override JSONB
effective_from nullable
effective_until nullable
created_at
updated_at

Exactly one scope must be populated:

```
CHECK (  
  num_nonnulls(  
    market_id,  
    package_id,  
    organization_id,  
    facility_id  
  ) = 1  
)
```

Resolution order:

Market workflow



Package workflow



Organization workflow



Facility workflow

The most specific active workflow assignment wins.

Example:

IN_CLINIC

- Search
- Registration
- Appointment
- Consultation
- Billing

UAE_HOSPITAL

- Search
 - Insurance Verification
 - Registration
 - Appointment
 - Triage
 - Consultation
 - Billing
-

7. **care** Schema

Purpose:

- appointments
- queue handling
- encounters
- core clinical records

care

- |— appointment
- |— queue_entry
- |— encounter
- |— vital_sign
- |— soap_note
- |— diagnosis
- |— prescription
- |— prescription_item

7.1 care.appointment

id
uuid
organization_id
facility_id
patient_id
practitioner_id nullable
scheduled_start
scheduled_end nullable
status
reason nullable
created_by_user_id
created_at
updated_at

Examples of appointment status:

scheduled
confirmed
checked_in
cancelled
completed
no_show

Appointment represents the intention to receive care.

It does not represent the actual clinical interaction.

7.2 **care.queue_entry**

Create only when token or queue management is part of the pilot.

id
uuid
organization_id
facility_id
appointment_id nullable
patient_id
queue_date
token_number
status
checked_in_at
created_at
updated_at

Constraint:

UNIQUE(facility_id, queue_date, token_number)

Examples of queue status:

waiting
called
in_consultation
completed
cancelled

7.3 **care.encounter**

Represents the actual healthcare interaction.

id
uuid
organization_id
facility_id
patient_id
practitioner_id
appointment_id nullable

status
started_at
ended_at nullable
created_at
updated_at

Appointment and encounter remain separate.

An appointment can:

- be cancelled
- become a no-show
- be rescheduled

without creating an encounter.

An encounter may also be created without an appointment for:

- walk-in patients
- emergency visits
- unscheduled consultations

7.4 care.vital_sign

id
organization_id
encounter_id
recorded_by_user_id
recorded_at
temperature nullable
pulse nullable
systolic_bp nullable
diastolic_bp nullable
spo2 nullable
weight nullable
created_at

Additional measurements can be added when required:

height
BMI

respiratory_rate
blood_glucose
pain_score

Avoid placing all vital values in one unstructured JSONB document if they will be searched, reported, or analyzed.

7.5 care.soap_note

id
uuid
organization_id
encounter_id
author_practitioner_id
subjective nullable
objective nullable
assessment nullable
plan nullable
status
created_at
updated_at

Examples of status:

draft
signed
amended
cancelled

Do not mix AI job execution state directly into the signed clinical record.

An AI-generated draft should eventually reference a separate AI job or generation record.

The signed clinical note remains the authoritative clinical record.

7.6 care.diagnosis

id

organization_id
encounter_id
code_system nullable
diagnosis_code nullable
description
is_primary
created_at
updated_at

Examples of `code_system`:

ICD-10
SNOMED-CT
local

A diagnosis can initially be entered as free text.

Standard coding can be introduced incrementally.

7.7 `care.prescription`

Prescription header.

id
uuid
organization_id
encounter_id
patient_id
practitioner_id
status
prescribed_at
created_at
updated_at

Examples of status:

draft
signed
cancelled
dispensed

7.8 **care.prescription_item**

One row per prescribed medicine.

id
prescription_id
medicine_name
dosage
frequency
duration nullable
instructions nullable
created_at
updated_at

Possible future columns:

medicine_code
dosage_unit
route
duration_unit
quantity

Create lab-order and procedure tables only when those workflows enter the product.

8. **financial** Schema

Purpose:

- patient billing
- payment collection

This remains separate from HealthOS SaaS subscription billing.

financial
├── bill
├── bill_item

|— payment
|— payment_allocation

8.1 financial.bill

id
uuid
organization_id
facility_id
patient_id
encounter_id nullable
bill_number
subtotal
discount_amount
tax_amount
total_amount
currency_code
status
created_at
updated_at

Constraint:

UNIQUE(organization_id, bill_number)

bill_number is a business identifier.

It must not be generated directly from the internal primary key.

Examples of bill status:

draft
issued
partially_paid
paid
cancelled
refunded

8.2 financial.bill_item

id
bill_id
description
quantity
unit_price
total_amount
created_at
updated_at

Possible future columns:

service_code
item_type
reference_type
reference_id
discount_amount
tax_amount

8.3 financial.payment

id
uuid
organization_id
facility_id
patient_id
amount
currency_code
payment_method
status
external_reference nullable
paid_at
created_at
updated_at

Examples of payment methods:

cash
card
UPI

bank_transfer
insurance
online_gateway

Examples of payment status:

pending
completed
failed
cancelled
refunded

8.4 financial.payment_allocation

Supports:

- partial payments
- one payment covering multiple bills
- one bill receiving multiple payments

id
payment_id
bill_id
allocated_amount
created_at

Constraint:

UNIQUE(payment_id, bill_id)

Create payment-gateway transaction tables only when online payments are implemented.

9. governance Schema

Purpose:

- healthcare audit trail
- patient consent
- lightweight compliance configuration

governance

- └─ audit_log
- └─ consent_record
- └─ compliance_policy

9.1 governance.audit_log

Append-only clinical and operational audit log.

id
organization_id
facility_id nullable
actor_user_account_id nullable
action
resource_type
resource_id
patient_id nullable
result
metadata JSONB
occurred_at

Examples of **action**:

patient.read
patient.create
patient.update
encounter.create
encounter.update
prescription.sign
report.export
consent.grant
consent.revoke
abdm.record_share

Examples of **result**:

allowed
denied
success
failure

Rules:

Application cannot update audit rows
Application cannot delete audit rows
Partition by occurred_at when volume grows

Logto records:

authentication activity
session activity
IAM activity
role assignment activity

HealthOS records:

clinical activity
patient-data access
billing activity
consent activity
feature usage
operational changes

9.2 governance.consent_record

id
uuid
organization_id
patient_id
consent_type
purpose nullable
status
granted_at nullable
expires_at nullable
revoked_at nullable
metadata JSONB

created_at
updated_at

Examples of consent type:

treatment
data_processing
telemedicine
record_sharing
ABDM
research
marketing

This can later store references to ABDM consent artifacts.

9.3 governance.compliance_policy

Create when the first ABDM, FHIR, DPDP, HIPAA, or similar requirement begins.

id
uuid
market_id
code
version
config JSONB
status
created_at
updated_at

Constraint:

UNIQUE(market_id, code, version)

Examples:

ABDM
FHIR_R4
DPDP
HIPAA

This table may later move to:

compliance.policy

At the initial stage, this table stores configuration and metadata.

It should not attempt to become a complete compliance automation engine.

10. **operations** Schema

Do not create this schema until the first notification or external integration is implemented.

Initial future structure:

```
operations
├── notification_outbox
├── integration_connection
└── integration_job
```

10.1 **operations.notification_outbox**

```
id
uuid
organization_id
channel
recipient
template_code
payload JSONB
status
attempt_count
scheduled_at
created_at
updated_at
```

Channels:

email
sms
whatsapp
push

This table supports asynchronous notification delivery.

It can later move to:

notification.outbox

10.2 operations.integration_connection

id
uuid
organization_id
provider
status
config JSONB
created_at
updated_at

Examples:

ABDM
payment_gateway
laboratory
WhatsApp
insurance_provider

Do not store secrets directly in JSONB.

Store secret-manager references.

Example:

```
{  
  "base_url": "provider-url",  
  "secret_reference": "vault://healthos/provider-key"  
}
```

10.3 **operations.integration_job**

id
uuid
organization_id
connection_id
job_type
reference_type
reference_id
status
payload JSONB
attempt_count
created_at
updated_at

Examples:

ABDM record push
FHIR bundle generation
lab result synchronization
payment webhook processing
WhatsApp message delivery

These tables can later move into dedicated:

notification
integration

schemas.

11. Tables Not Required Initially

Do not create these until there is a concrete product requirement:

organization_group
commercial_usage_meter

workflow_execution
access_grant
break_glass_event
lab_order
lab_order_item
procedure_record
payment_transaction
inventory tables
notification delivery history
FHIR mapping tables
ABDM exchange tables
AI job tables
analytics tables

The architecture reserves space for these capabilities without requiring their implementation in the first version.

12. Initial Table Count

Create immediately:

identity	4 tables
organization	5 tables
platform	8 tables
care	7 tables
financial	4 tables
governance	2 tables

Total:

30 tables

Optional additions:

care.queue_entry	+1
governance.compliance_policy	+1 when required
operations schema	when required

13. Runtime Authorization and Product Resolution

When a user opens HealthOS:

1. Validate the Logto JWT.
2. Resolve identity.user_account using the Logto "sub".
3. Resolve the selected Logto organization.
4. Resolve organization.organization using logto_organization_id.
5. Resolve organization.staff_member for the user and tenant.
6. Verify an active organization.staff_assignment for the selected facility.
7. Resolve the organization's market.
8. Resolve the active commercial subscription and package.
9. Resolve market feature assignments.
10. Apply package feature assignments.
11. Apply organization overrides.
12. Apply facility overrides.
13. Resolve the effective workflow assignment.
14. Verify Logto roles and permissions.
15. Apply healthcare-specific contextual checks.
16. Record the operation in governance.audit_log.
17. Return the frontend product context.

The effective product is:

HealthOS Product

=

Market defaults

+

Package entitlements

+

Organization overrides

+

Facility overrides

Authorization is:

Access

=

Valid Logto identity

+

Logto permission
+
Active HealthOS staff membership
+
Valid facility assignment
+
Enabled feature
+
Healthcare-specific contextual rule

14. Frontend Context Endpoint

Recommended endpoint:

GET /api/platform/context

Example response:

```
{
  "market": {
    "code": "IN_CLINIC",
    "currency": "INR",
    "timezone": "Asia/Kolkata"
  },
  "package": {
    "code": "IN_GROWTH"
  },
  "organization": {
    "uuid": "organization-public-uuid"
  },
  "facility": {
    "uuid": "facility-public-uuid"
  },
  "features": {
    "patient_registration": {
      "enabled": true
    },
    "queue_management": {
      "enabled": true
    }
  },
}
```

```
"ai_soap": {
  "enabled": false
},
"abha_linking": {
  "enabled": true
}
},
"workflows": {
  "patient_intake": {
    "version": 1,
    "steps": [
      "patient.search",
      "patient.registration",
      "appointment.booking"
    ]
  }
}
}
```

The frontend uses this response for rendering.

The backend independently enforces the same:

- feature rules
- tenant rules
- facility rules
- authorization rules
- contextual healthcare rules

15. Essential Database Rules

From the beginning:

Use BIGINT identity keys for internal joins

Use UUIDs for public and external references

Use foreign keys

Use TIMESTAMPTZ and UTC

Use organization_id on tenant-owned data

Use facility_id where the record is facility-specific

Use tenant-scoped unique constraints

Use INSERT ... RETURNING or ORM flush to retrieve generated IDs

Do not use SELECT MAX(id)

Do not expose internal incremental IDs through public APIs

Do not use IDs as business sequence numbers

Do not store Logto passwords, sessions, roles, or permissions

Do not use email as the authoritative Logto identity

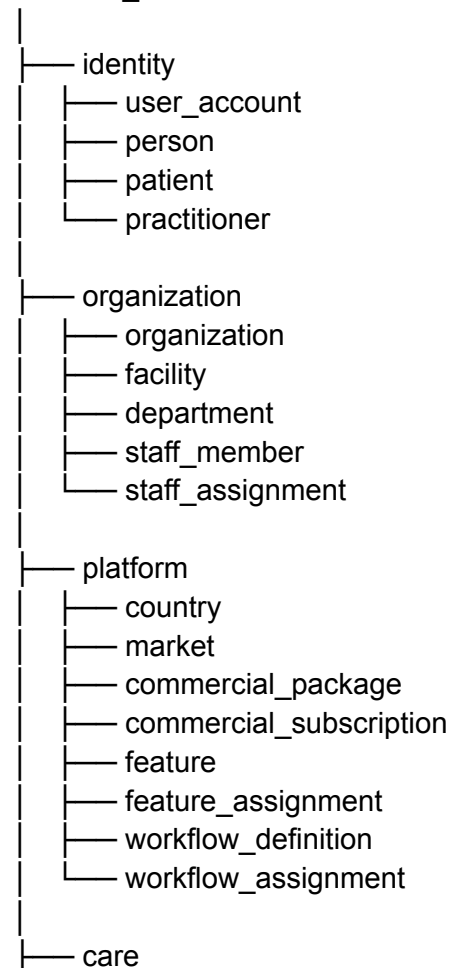
Do not create one schema per hospital

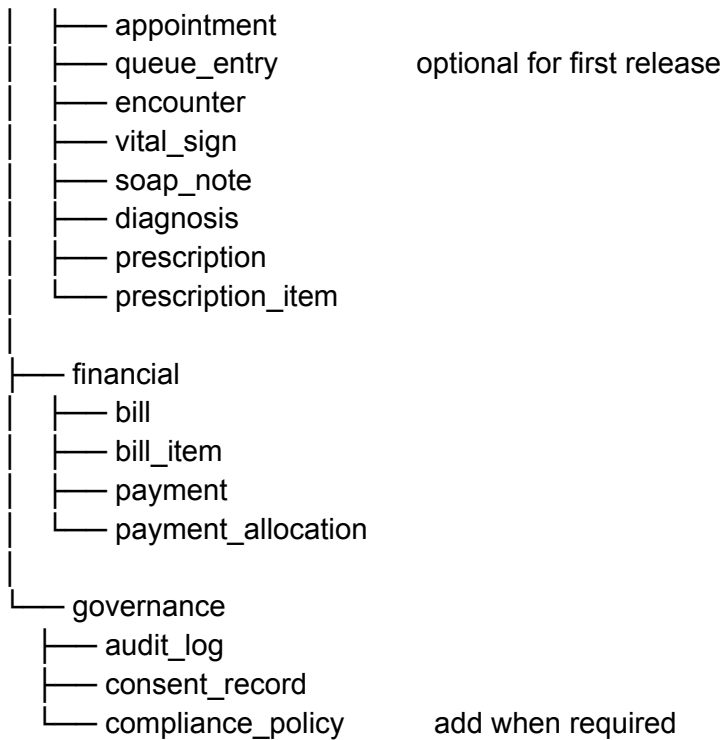
Do not store core clinical records as large JSONB documents

Add PostgreSQL Row-Level Security after the request-scoped organization context is implemented and tested reliably.

16. Final Initial Schema

healthos_db





17. Final Architecture Coverage

This baseline supports:

Logto authentication
Logto generic RBAC
front-desk and non-clinical staff
clinical practitioners
patients with or without accounts
multi-tenant organizations
multiple facilities
department assignments
market-specific products
package-based licensing
organization overrides
facility overrides
dynamic feature resolution
configurable frontend workflows
efficient BIGINT joins
public UUID identifiers
clinical operations

- billing and payment allocation
- patient consent
- clinical audit logging
- future ABDM integration
- future FHIR integration
- future AI modules
- future notification and integration services

This is the recommended initial implementation baseline for HealthOS.